

April 8th, 2026

Improving Scientific Software 2026

Boulder, CO, USA

ALWAYS MEASURE

Analyzing Performance of Point-Particle DNS under Two-Way
Momentum Coupling

Maciej MANNA

maciej.manna@imgw.pl

*Institute of Meteorology and Water Management – National Research Institute (IMGW-PIB)
Kraków, Poland*

with **Bogdan ROSA**
and **Ahmad ABABAEI**



BACKGROUND

ME AND RESEARCH SOFTWARE:

Software Engineer in the Industry

- 8+ years of experience as developer and Tech Lead
- mainly C++ and C#
- from startups to large established software companies



WINCAN



AUTODESK

Recently switched (back) to Research

- Earlier internship as part of MSc thesis preparation
- now research job and working towards PhD
- research area: numerical study collision and coalescence of cloud droplets in atmospheric turbulence



BACKGROUND

ME AND RESEARCH SOFTWARE:

Software Engineer in the Industry

- 8+ years of experience as developer and Tech Lead
- mainly C++ and C#
- from startups to large established software companies



WINCAN



AUTODESK

Recently switched (back) to Research

- Earlier internship as part of MSc thesis preparation
- now research job and working towards PhD
- research area: numerical study collision and coalescence of cloud droplets in atmospheric turbulence



ME AND ISS 2025:

- Being on-site last year (accompanying my partner), great experience
- No presentation of my own : (
- Wanted to share some previous results, especially now – going back to research

SCIENTIFIC SOFTWARE AND PERFORMANCE

WE CARE ABOUT PERFORMANCE OF OUR CODE

- saving precious resources on HPC platforms we have at our disposal,
- running software used in day-to-day operations (e.g. weather prediction) in predetermined slots and producing results in time,
- being able to run research code on a laptop instead of a HPC resources,
- (at least for some of us) it is fun in itself :)

MAIN OBJECTIVE

THE REALITY OF MODERN SOFTWARE IS COMPLEX:

COMPLEX
HARDWARE

COMPLEX
TOOLS AND SOFTWARE
(compilers, etc.)

COMPLEXITY OF
RESEARCH PROBLEMS

MAIN OBJECTIVE

THE REALITY OF MODERN SOFTWARE IS COMPLEX:

COMPLEX
HARDWARE

COMPLEX
TOOLS AND SOFTWARE
(compilers, etc.)

COMPLEXITY OF
RESEARCH PROBLEMS

OUR **INTUITIONS** ABOUT THE PERFORMANCE OF OUR CODE
CAN BE **HEURISTICS** AT BEST

THE ONLY WAY TO RELIABLY **VALIDATE** OUR CLAIMS ABOUT PERFORMANCE
IS TO PROPERLY **MEASURE** IT

AGENDA

(slides available at: github.com/xann16/talks)

QUICK REFRESHER ON SOFTWARE PERFORMANCE

(basic concepts, benchmarking and profiling)

PERFORMANCE AND MODERN COMPILERS / HARDWARE

(showcase how modern optimizing compilers and CPU architecture often challenge our intuitions about code performance)

RESEARCH SOFTWARE CASE STUDY – EXTRA COMPLEXITIES

(example of performance measurements for massively parallel code for simulating cloud droplets in turbulent flow, Its complexities, observations challenging intuitive assumptions)

WHAT IS SOFTWARE PERFORMANCE?

A lot of broader and narrower meanings of PERFORMANCE in software...

In narrower sense, how effectively our software uses available resources, e.g.:

- CPU time / cycles,
- memory,
- network bandwidth,
- power (electricity),
- heterogeneous architecture resources (e.g. GPU, etc.),
- ...

WHAT IS SOFTWARE PERFORMANCE?

A lot of broader and narrower meanings of PERFORMANCE in software...

In narrower sense, how effectively our software uses available resources, e.g.:

- CPU time / cycles,
 - memory,
 - network bandwidth,
 - power (electricity),
 - heterogeneous architecture resources (e.g. GPU, etc.),
 - ...
- “Computational performance” or “computational efficiency”, two aspects:
- **Throughput** (as much computation as possible in given time),
 - **Latency** (stable response times, highly controlled variance).

WHAT IS SOFTWARE PERFORMANCE?

A lot of broader and narrower meanings of PERFORMANCE in software...

In narrower sense, how effectively our software uses available resources, e.g.:

- CPU time / cycles,
 - memory,
 - network bandwidth,
 - power (electricity),
 - heterogeneous architecture resources (e.g. GPU, etc.),
 - ...
- “Computational performance” or “computational efficiency”, two aspects:
- Throughput (as much computation as possible in given time),
 - Latency (stable response times, highly controlled variance).

CPU PERFORMANCE METRICS

Certain metrics may provide some **absolute** measurements of computational performance in terms of number of CPU cycles, and other useful information (see e.g. Linux `perf` tool).

For the most part, the most reliable and useful measurement are in terms of **relative** execution time. We often use following metrics (T – execution times):

$$\text{SPEEDUP} = T_{\text{base}} / T_{\text{new}}$$

$$\% \text{ IMPROVEMENT} = (1 - (T_{\text{new}} / T_{\text{base}})) * 100\%$$

CPU PERFORMANCE METRICS

Certain metrics may provide some **absolute** measurements of computational performance in terms of number of CPU cycles, and other useful information (see e.g. Linux `perf` tool).

For the most part, the most reliable and useful measurement are in terms of **relative** execution time. We often use following metrics (T – execution times):

$$\text{SPEEDUP} = T_{\text{base}} / T_{\text{new}}$$

$$\% \text{ IMPROVEMENT} = (1 - (T_{\text{new}} / T_{\text{base}})) * 100\%$$

NOTE: In HPC / parallelized context we may care either about (1) total execution time, or (2) total time scaled by the number of resources used (e.g. parallel nodes) to estimate resource usage.

PERFORMANCE MEASUREMENTS

Two types of methods for measuring performance include **(micro-)benchmarks** and **profiling**.

BENCHMARKS *(separate evaluation of small, often critical section of code)*

- + good isolation of code of interest
- + good tooling, convenient to obtain statistically significant results

- not always representative of real-world conditions

PROFILING *(measurement of real software as used, either by instrumentation or statistical sampling)*

- + representative of actual code in use, in the context of other components, OS and hardware resources availability
- + good generic tools that allow to identify hot spots

- instrumentation or sampling may affect performance and reliability of measurements
- points of interest or causes of performance issues may not be easily distinguishable due to external noise

COMPLEXITY OF MODERN SOFTWARE AND HARDWARE

OUR
CODE

Optimizing compilers / JIT-compilers / Interpreters

External Libraries

OS Resource Management

Modern Hardware Architecture (multiple cores, caches, memory access)

Modern CPU Execution (branch prediction, pipelining, speculative execution)

“EXECUTION”

COMPILER OPTIMIZATION

Modern compilers are **AMAZING** at optimizing our code (for compiled, JIT-compiled languages).

See short series on YT by Matt Godbolt: “Advent of Compiler Optimisations 2025”!

IMPORTANT – GIVE YOUR COMPILER AS MUCH INFORMATION AS POSSIBLE:

- about the code itself – example: compile-time values or array sizes, if possible (compiler may do a lot of micro-optimizations for You, decide about inlining, loop unrolling, etc.)
- about target architecture – important for research and scientific code as its execution is often limited to certain hardware or recompiled before use; it may allow compiler to be less conservative and use specific instructions available, e.g. better SSE instructions for vectorization of computations

COMPILER OPTIMIZATION

Modern compilers are **AMAZING** at optimizing our code (for compiled, JIT-compiled languages).

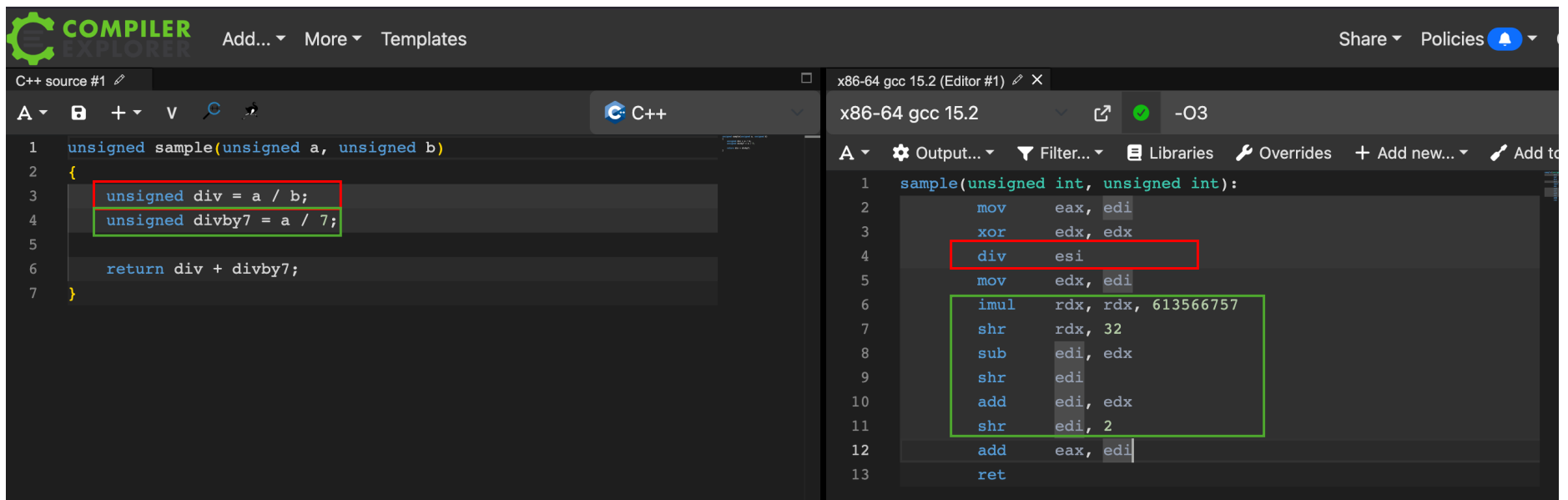
See short series on YT by Matt Godbolt: “Advent of Compiler Optimisations 2025”!

IMPORTANT – GIVE YOUR COMPILER AS MUCH INFORMATION AS POSSIBLE:

- about the code itself – example: compile-time values or array sizes, if possible (compiler may do a lot of micro-optimizations for You, decide about inlining, loop unrolling, etc.)
- about target architecture – important for research and scientific code as its execution is often limited to certain hardware or recompiled before use; it may allow compiler to be less conservative and use specific instructions available, e.g. better SSE instructions for vectorization of computations

Remember about flag: `--march`

COMPILER OPTIMIZATION



The image shows a screenshot of the Compiler Explorer interface. On the left, the C++ source code is displayed in a dark-themed editor. The code defines a function `sample` that takes two unsigned integers `a` and `b` and returns their sum. Two intermediate variables, `div` and `divby7`, are calculated. The `div` variable is assigned the value of `a / b`, and `divby7` is assigned the value of `a / 7`. The final return value is `div + divby7`. The `div` assignment and the `divby7` assignment are highlighted with a red box. The `divby7` assignment is also highlighted with a green box. On the right, the assembly output for the same code is shown. The assembly is for x86-64 gcc 15.2, compiled with `-O3`. The assembly code shows the function `sample` taking two unsigned integers. It starts by moving `edi` into `eax`, then XORs `edx` with `edx`. The `div esi` instruction is highlighted with a red box. This is followed by moving `edi` into `edx`, then multiplying `rdx` by the constant `613566757`. The result is then shifted right by 32 bits, subtracted from `edi`, shifted right by 1 bit, added to `edx`, and shifted right by 2 bits. Finally, `eax` is added to `edi`, and the function returns.

```
1 unsigned sample(unsigned a, unsigned b)
2 {
3     unsigned div = a / b;
4     unsigned divby7 = a / 7;
5
6     return div + divby7;
7 }
```

```
1 sample(unsigned int, unsigned int):
2     mov     eax, edi
3     xor     edx, edx
4     div     esi
5     mov     edx, edi
6     imul    rdx, rdx, 613566757
7     shr     rdx, 32
8     sub     edi, edx
9     shr     edi
10    add     edi, edx
11    shr     edi, 2
12    add     eax, edi
13    ret
```

INTRICACIES OF MODERN CPU ARCHITECTURE

Some important aspects of modern CPU execution:

- caches (several levels of cache, both for data and instructions)
- pipelining
- branch prediction
- speculative code execution
- ...

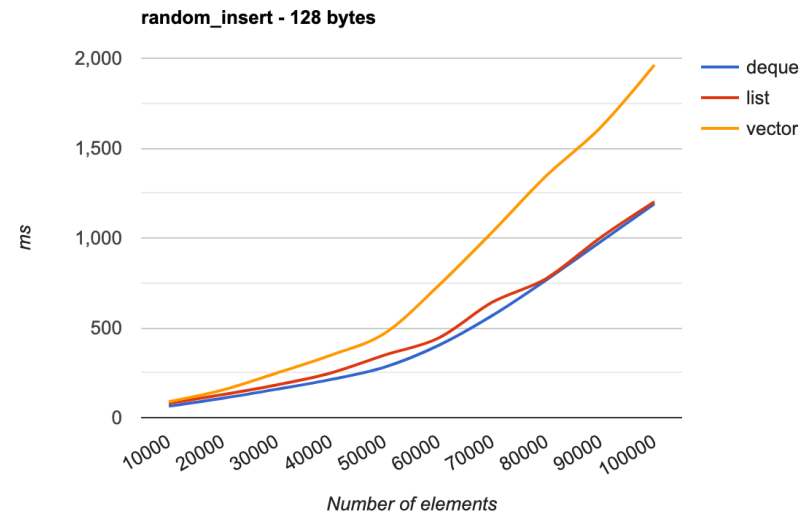
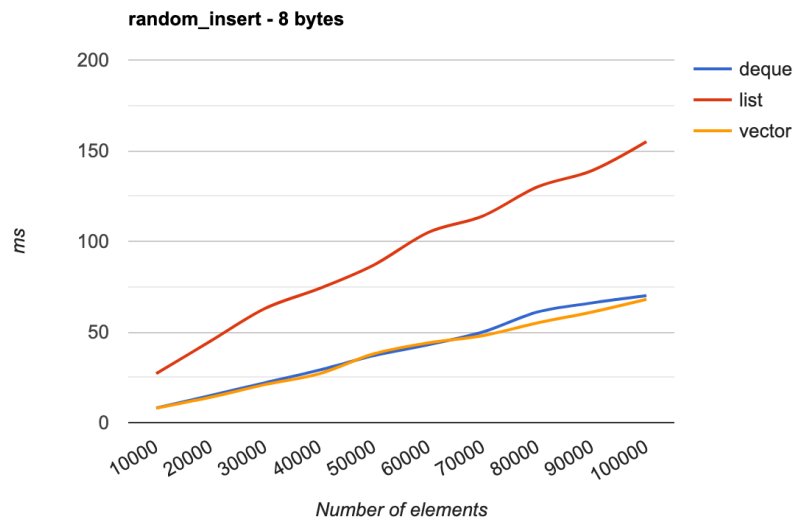
In general, (1) code that has predictable access patterns, and (2) operates on contiguous data is much easier to optimize both for software (compiler) and hardware (internal CPU scheduling and optimization facilities).

THEORY VS PRACTICE – CLASSIC EXAMPLE

Two data structures: dynamically-sized contiguous array (C++: `std::vector`) and double-linked list (C++: `std::list`).

The latter has much better theoretical performance for insertions and deletions ($O(1)$), while the former – in the worst case – needs to copy all its elements to maintain contiguity ($O(n)$).

THEORY VS PRACTICE – CLASSIC EXAMPLE



Sources: <https://baptiste-wicht.com/posts/2012/12/cpp-benchmark-vector-list-deque.html>
https://fylux.github.io/2017/06/29/list_vs_vector/

ADDITIONAL COMPLEXITIES RELATED TO RESEARCH SOFTWARE

All previous observations apply, sometimes even are more glaring when we work with numeric code.

Some additional important topics:

- complexity associated with **massive parallelism and distributed execution** across multiple threads, processes, or using accelerators (GPU, TPU, ...),
- emergent complexity associated with **physical systems** being modelled that may not be fully accounted for when designing numeric simulations.

CASE STUDY...

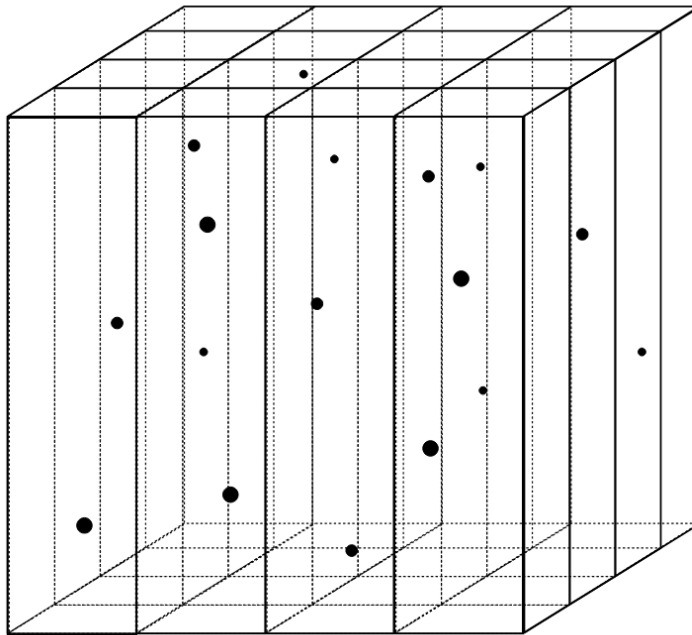
CASE STUDY

Numerical simulations to estimate collision statistics of droplets in turbulent flow under two-way momentum coupling. Specific goal: establish whether LES may be a viable alternative for DNS to resolve fluid flow (to obtain higher Reynolds number with same computational cost).

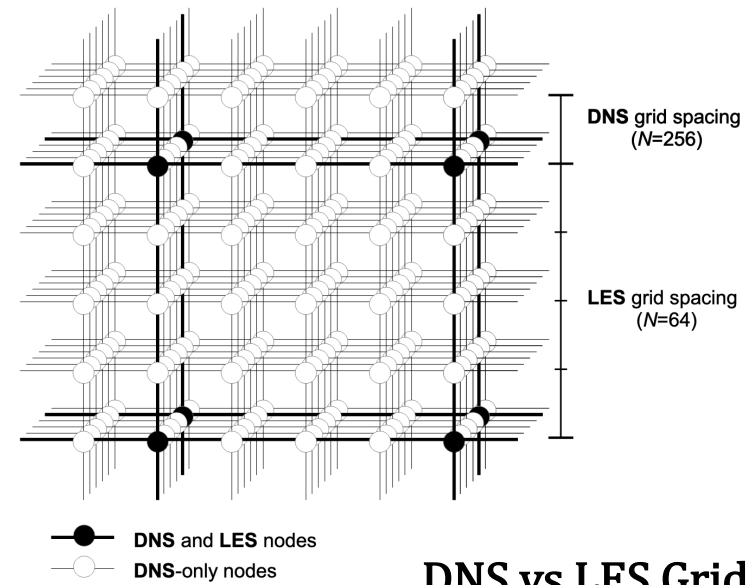
- combination (with two-way momentum transfer) of:
 - Eulerian grid of flow field with pseudo-spectral N-S solver with external forcing to maintain stationary homogeneous and isotropic turbulence (DNS or LES using spectral-eddy viscosity – Smagorinsky model),
 - Individual Lagrangian particles;
- 2D domain decomposition (into “pencils”) of Eulerian grid (keeping particles in subdomains where they are located with respect to grid) – multi-process parallelisation with communications using MPI;
- statistics processing code – mainly related to particle pairs (collisions and collision kernel, RDF, RRV).

More details: <http://data.xann16.eu/thesis.pdf>

CASE STUDY

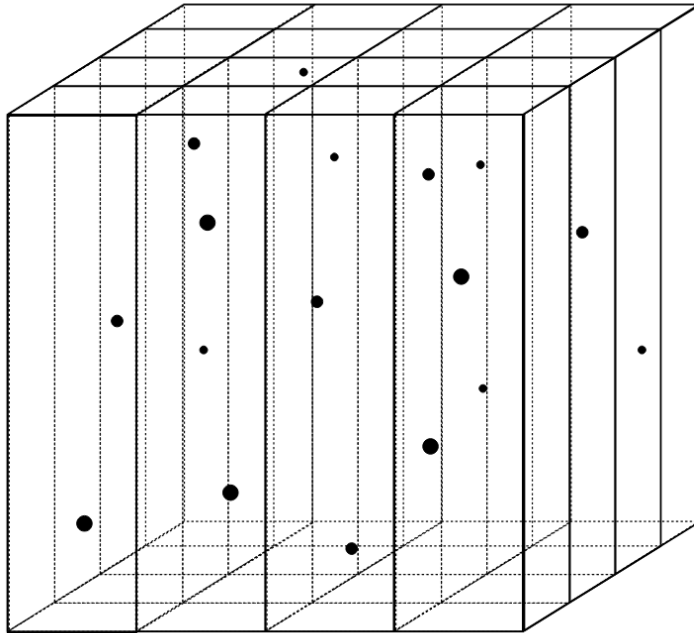


2D Domain Decomposition



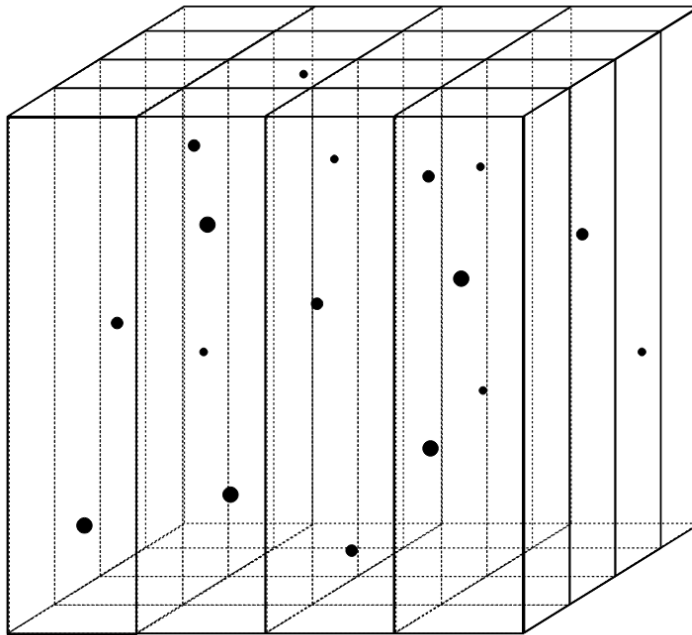
DNS vs LES Grid

WHY “PENCILS”?



2D Domain Decomposition

WHY “PENCILS”?



2D Domain Decomposition

Other options:

- 1D slabs – reduced potential for parallelism,
- 3D boxes – [measurements](#) have shown that additional costs of inter-process communication overshadows any gains from additional parallelization.

Also, main workhorse of pseudo-spectral solver is 3D Fast Fourier Transform (FFT), that is composed of 1D FFTs with intermediate data transposition. This makes pencils suitable, as all data for 1D FFTs is contained within single subdomain.

See: Dmitruk et al., *Parallel Computing* (2001).

SIMPLE “PROFILING” STRATEGY

- Simple manual instrumentation – timing execution of each iteration at several points (based on its general structure – see right);
- By default 200000 iterations per simulation (enough to reduce variance, data from some initial and final iterations discarded);
- Total simulation times ranged from 0.5h to 15 days – average iteration time between 9ms and 6s (should not be significantly affected by this amount of minimal instrumentation).

No. (Cat.)	Name Description	Interdomain communication	Notes
1 (F)	Transform $\hat{u}$ and $\hat{\omega}$ to physical space <i>Uses inverse 3D FFT twice; first two steps of pseudo-spectral method procedure.</i>	FFT	Eqn. A.6 and A.7.
2 (P)	Calc. momentum transfer from fluid to particles <i>Finds closest grid points to particles and calculates momentum transfer using six-point Lagrange interpolation scheme (OWC).</i>	Local	—
3 (P)	Calc. momentum transfer from particles to fluid <i>Uses PNN kernel to apply momentum transfer projected from particles onto fluid at neighbouring grid points (TWC).</i>	Local	Eqn. 1.8. (TWC only)
4 (P)	Calc. new particle velocities and positions <i>Integrates equations of motion for particles to obtain their new velocities and positions; updates data structures used to keep track of neighbouring particles.</i>	Local	Eqn. 1.5.
5 (Pd)	Calculate and write particle collision statistics <i>Detects and counts particle collisions (Γ_{11}^P) and relative particle-pair data within radial shells necessary to compute RDF and RRV.</i>	Global (only to write data), Local	Sec. 2.2.1 and 2.2.2; every step; for steps > 30000 (after system relaxation).
6 (P)	Update particle velocities and positions <i>Changes the state of particles as calculated in step 4; transfers data for particles crossing subdomain boundaries.</i>	Global, Local	—
7 (Fd)	Calculate and write flow statistics and spectra <i>Obtains and stores energy and dissipation spectra; calculates other base and derived flow statistics (u' and ϵ).</i>	Global, FFT, Local	Sec. 2.1.1 and 2.1.2; every 50 steps*.
8 (F)	Transform $u \times \omega$ to spectral space <i>Uses 3D FFT to perform third step of pseudo-spectral method procedure; performs post-transform dealiasing.</i>	FFT	Eqn. A.8.
9 (F)	Calc. effective viscosity using subgrid-scale model <i>Uses fluid energy spectrum and subgrid-scale model formulation to adjust kinematic viscosity with spectral-eddy viscosity, i.e. $\nu + \nu_e(k k_c)$.</i>	Global	Eqn. 1.3 and 1.4. (LES only)
10 (F)	Integrate fluid flow equations <i>Integrates 3D Navier-Stokes equation using Crank-Nicholson scheme (fourth step of pseudo-spectral method); incorporates deterministic forcing and other external forces; performs necessary dealiasing and symmetrisation.</i>	Global, Local	Eqn. A.12

SIMPLE RESULTS SNAPSHOT – EXPECTED

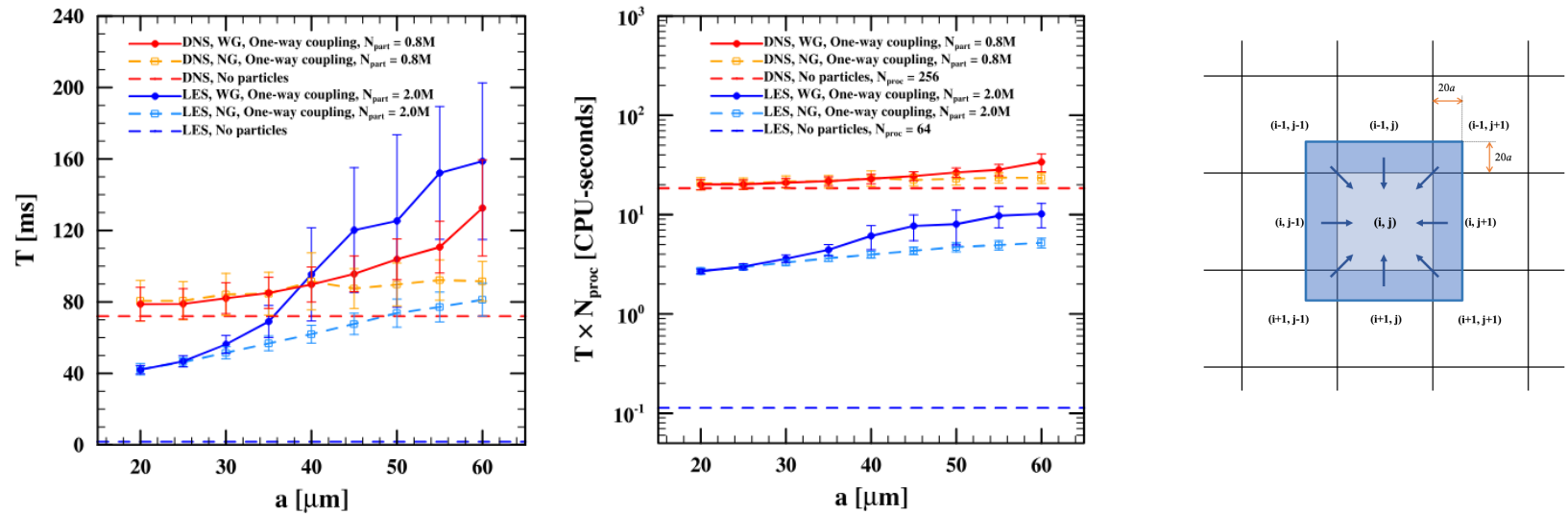


Figure 3.2: Time-averaged wall-clock times per step by itself (left) and multiplied by number of parallel processes used to execute code (right, in log scale) for simulations under one-way momentum coupling (OWC) presented as a function of particle radii. Both results for DNS and LES are presented, with (WG) and without (NG) gravity. Horizontal dashed lines represent values for simulations without particles.

SIMPLE RESULTS SNAPSHOT – EXPECTED

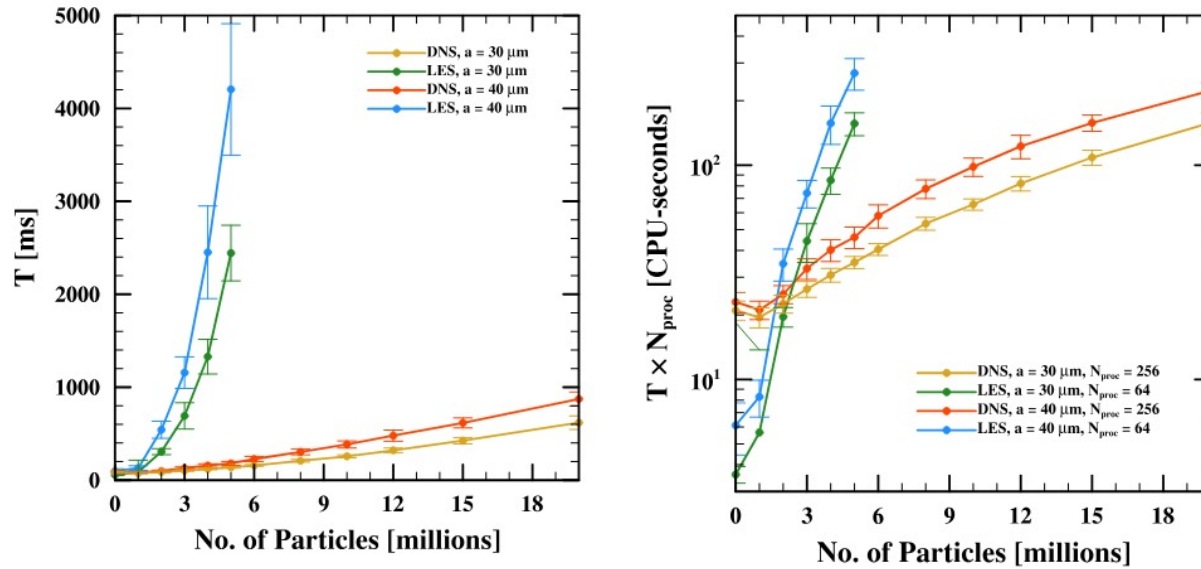


Figure 3.3: Time-averaged wall-clock times per step by itself (left) and multiplied by number of parallel processes used to execute code (right, in log scale) for simulations under two-way momentum coupling (TWC, with gravity) as a function of the number of computational particles. Results for both DNS and LES are presented.

SIMPLE RESULTS SNAPSHOT – EXPECTATIONS CONFIRMED

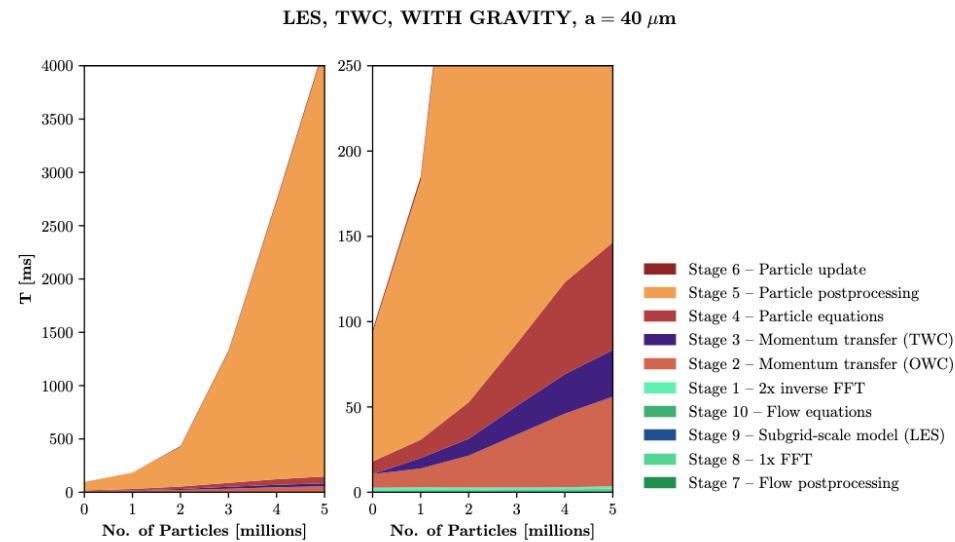


Figure 3.8: Performance profile for TWC LES simulations with gravity for droplets with $40 \mu\text{m}$ radius. Consistent with Figure 3.6. Both plots present profiles for the same range of simulations but the one on the right focuses on smaller span of values on the y -axis to “magnify” data pertaining to timings other than $T_5 (= T_{Pd})$ that dominates plot on the left. The leftmost profile (for 0 on x -axis) was provided for corresponding OWC simulation with $N_{\text{part}} = 800\text{k}$.

SIMPLE RESULTS SNAPSHOT – EXPECTATIONS CONFIRMED

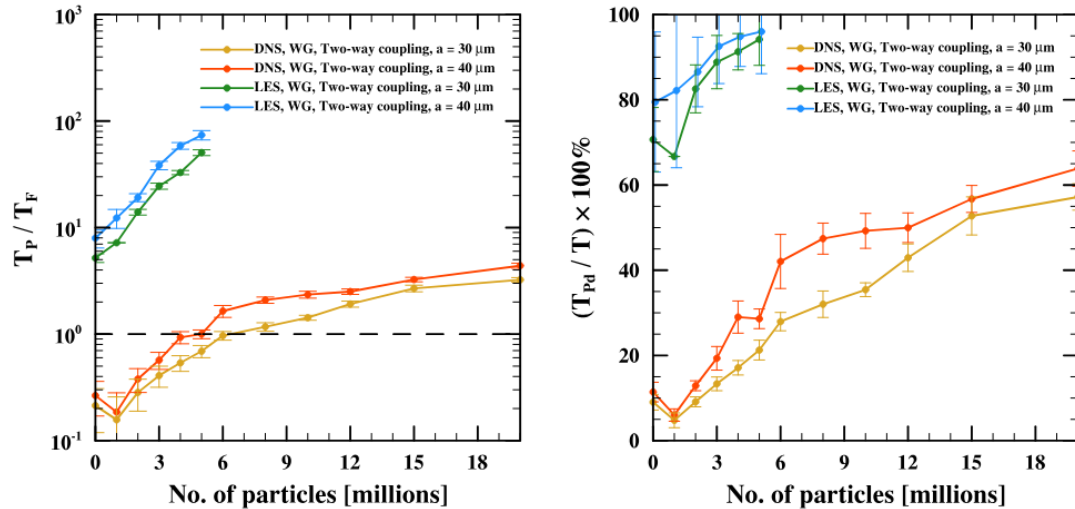


Figure 3.10: The ratio of average wall-clock times spent on processing particles, T_P , and the fluid flow, T_F (left, linear-log scale); and average percentage of overall step time occupied by particle postprocessing tasks, (T_{Pd}/T) (right). Both statistics are shown for TWC simulations with gravity and. Plots for LES on the right have slight horizontal offset added to make uncertainty bars more visible. Consistent with Figure 3.7.

ANALYZING PARTICLE DISTRIBUTION ACROSS SUBDOMAINS

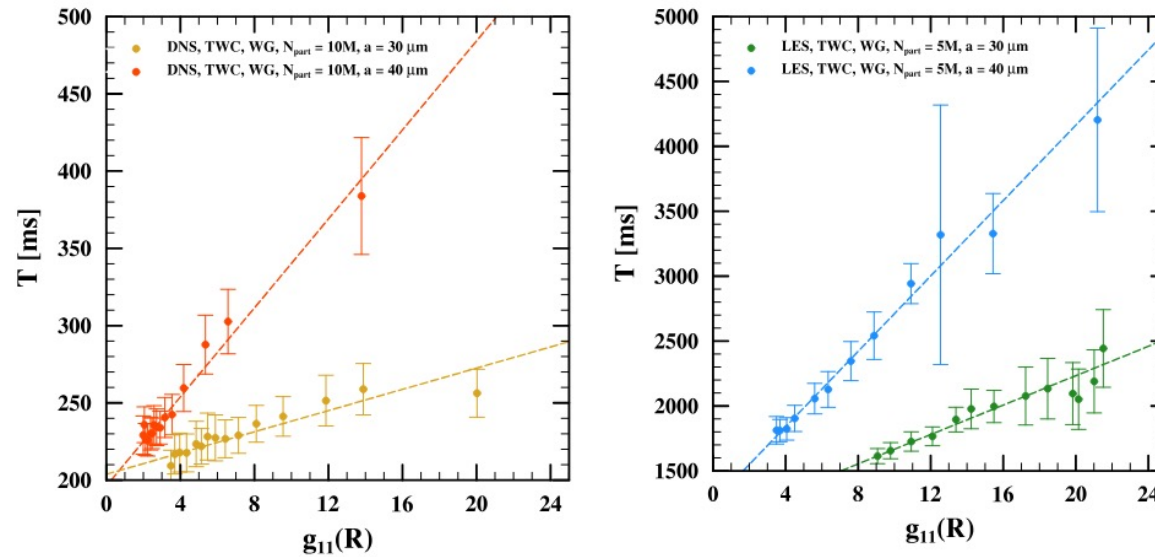
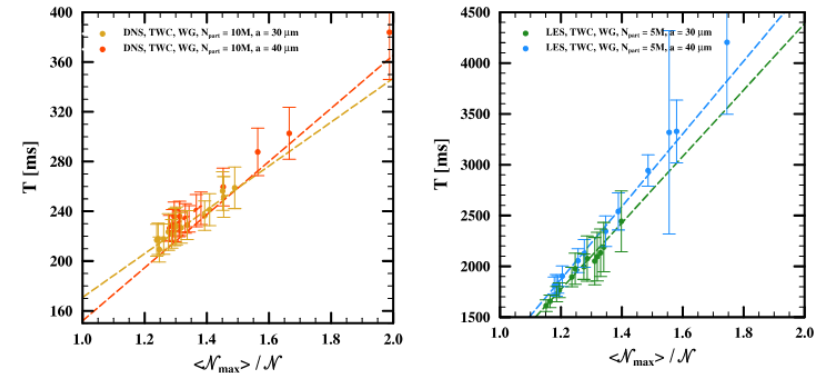
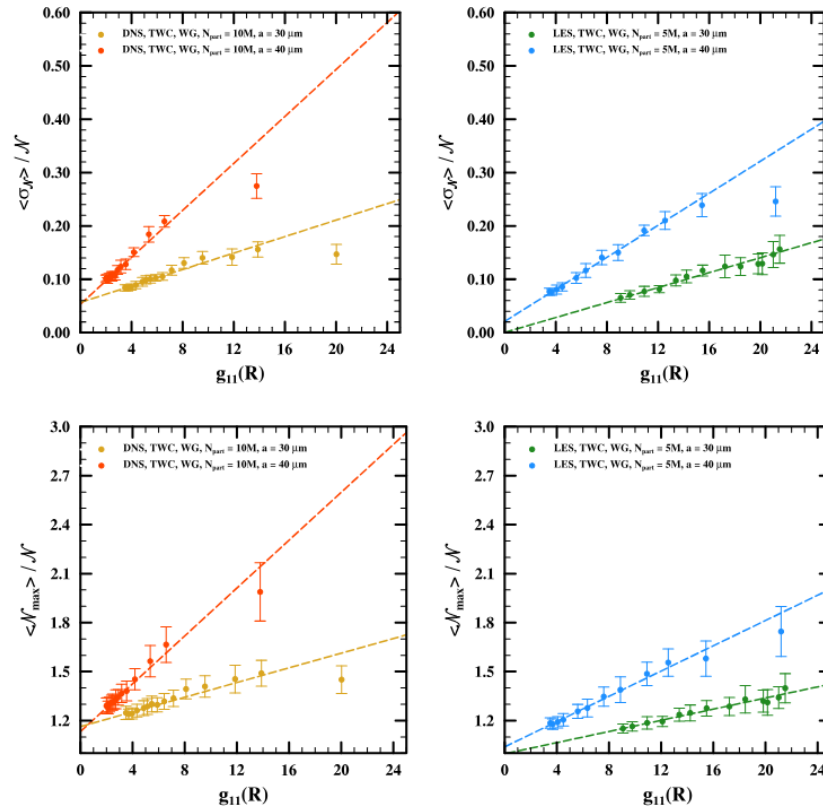


Figure 3.5: Time-averaged wall-clock times per step for DNS (left) and LES (right) simulations under two-way momentum coupling (TWC) with fixed number of computational particles (10 million for DNS, 5 million for LES) and radii ($30\text{ }\mu\text{m}$ and $40\text{ }\mu\text{m}$), depending on the estimated values of the radial distribution function at contact range (RDF at $r = R$). Dashed lines represent linear fit of included results weighted by the inverse variance of respective data points.

ANALYZING PARTICLE DISTRIBUTION ACROSS SUBDOMAINS

- RDF – a continuous, statistical, and physics-based statistics used to quantify the preferential concentration of particles – shows clear correlation with purely technical wall-clock time of simulation,
- to establish connection, extra data was collected – i.e. number of particles in each subdomain throughout the simulation,
- both simulation time and RDF also shown correlation with the (averaged over time) standard deviation of the distribution of particles across subdomains (purely discrete and technical statistic); similar correlation exist with (averaged in time) maximal recorded particle count in the single subdomain.

ANALYZING PARTICLE DISTRIBUTION ACROSS SUBDOMAINS



ANALYZING PARTICLE DISTRIBUTION ACROSS SUBDOMAINS

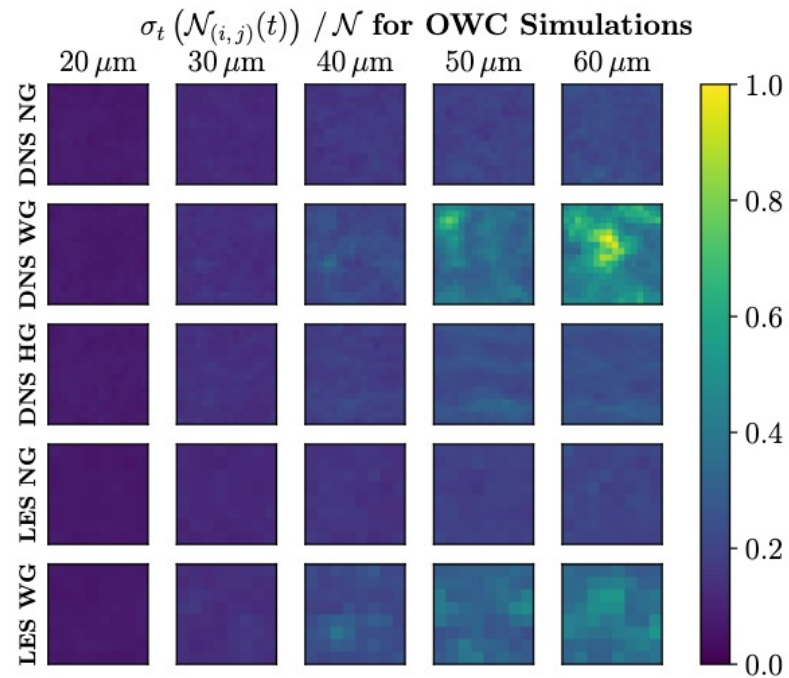


Figure 3.19: Standard deviation of particle counts for every subdomain taken over the entire run for OWC simulations with droplets of different radii. Consistent with Figure 3.19.

“HORIZONTAL GRAVITY”

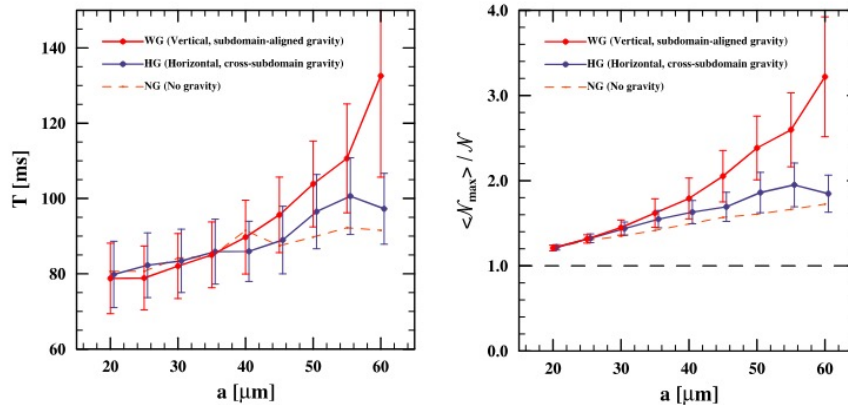


Figure 3.21: Time-averaged wall-clock times per step, T (left), and normalised maximal number of particles in subdomains, $\langle N_{\max} \rangle / N$ (right) for DNS simulations under one-way momentum coupling (OWC) with different directions of gravity (relative to the direction of domain decomposition). Results of simulations without gravity are shown with dashed lines for comparison.

CHANGE DIRECTION OF GRAVITY:

- direction of gravity changed from “along the pencils” to “across the pencils”,
- original rationale (see Ayala et al., *CPC*, 2014) – assuming homogeneous distribution of particles the direction “along the pencils” was chosen to minimize necessity to transfer particles between subdomains,
- however, such assumption is invalid, due to concentrated structures of large-inertia particles,
- this change amounts to changing 2 lines of code, and does not bring significant deterioration of performance, but in specific conditions (larger particles) may reduce execution time by about 30%

“HORIZONTAL GRAVITY”

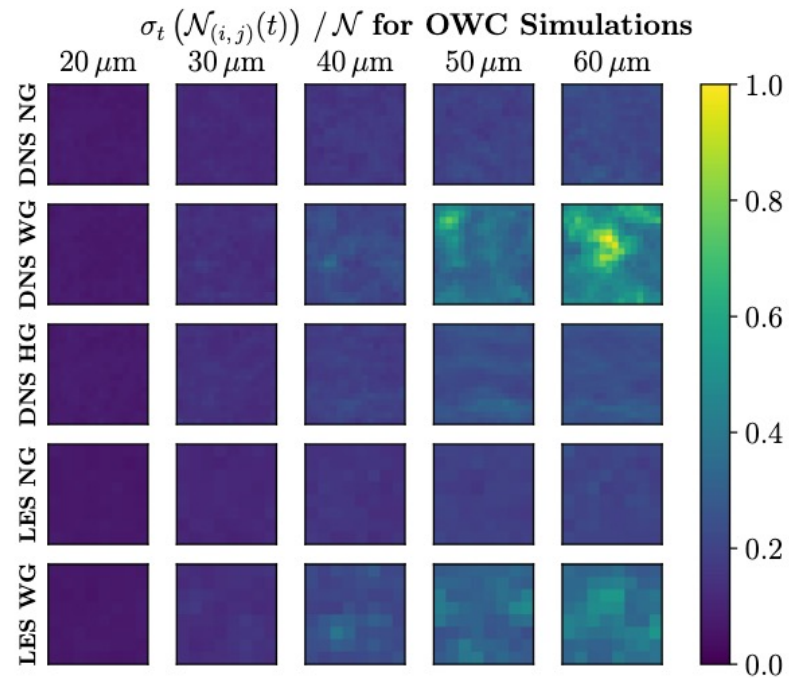
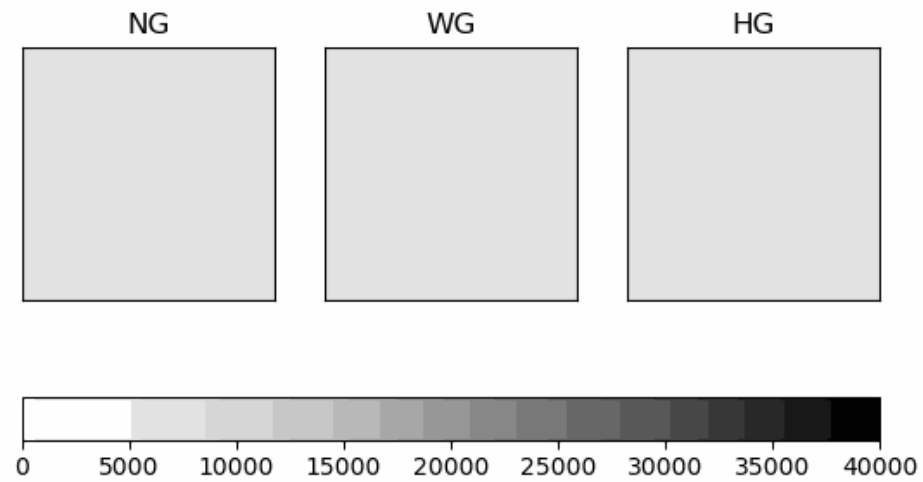


Figure 3.19: Standard deviation of particle counts for every subdomain taken over the entire run for OWC simulations with droplets of different radii. Consistent with Figure 3.19.

“HORIZONTAL GRAVITY”

step: 0; time: 0.0000 s; (time / T_e): 0.0000



ALWAYS MEASURE!

THE REALITY OF MODERN SOFTWARE IS COMPLEX:

COMPLEX
HARDWARE

COMPLEX
TOOLS AND SOFTWARE
(compilers, etc.)

COMPLEXITY OF
RESEARCH PROBLEMS

OUR **INTUITIONS** ABOUT THE PERFORMANCE OF OUR CODE
CAN BE **HEURISTICS** AT BEST
THE ONLY WAY TO RELIABLY **VALIDATE** OUR CLAIMS ABOUT PERFORMANCE
IS TO PROPERLY **MEASURE** IT

ALWAYS MEASURE!



THANK YOU!

Contact me: maciej.manna@imgw.pl

Slides can be found at:

<https://github.com/xann16/talks>

More information on case study project:

<https://data.xann16.eu/thesis.pdf>

Remember to check :

“Advent of Compiler Optimisations 2025”

By Matt Godbolt